

გაკვეთილი 2 Mod 7 (7.1)

გაკვეთილი 2 — Git-ში ცვლილებების შენახვა: `add` , `commit` , `log`

თემა: პროექტში განხორციელებული ცვლილებების კონტროლი და ვერსიების შექმნა

ხანგრძლივობა: 40 წუთი

წინა გაკვეთილიდან: მოსწავლეებმა უკვე იციან, რა არის Version Control, Git, GitHub, Repository, `git init` , `git status` და Untracked file.

გაკვეთილის მიზანი: მოსწავლემ გაიგოს, როგორ გადის ფაილი Git-ის სამუშაო პროცესს და პრაქტიკულად შეძლოს ცვლილებების მომზადება და შენახვა `git add` და `git commit` ბრძანებებით.

1. წინა გაკვეთილის მოკლე გამოვლი

გაკვეთილის დასაწყისში ვკითხოთ:

რა არის **Git**?

Git არის ვერსიების კონტროლის სისტემა, რომელიც პროექტში განხორციელებული ცვლილებების ისტორიას ინახავს და მართავს.

რა არის **Repository**?

პროექტი/საქაღალდე, რომლის ცვლილებებსაც Git აკონტროლებს.

რას აკეთებს `git init` ?

```
git init
```

ჩვეულებრივ საქაღალდეში იწყებს Git-ის ვერსიების კონტროლს და ქმნის Git Repository-ს.

რას აკეთებს `git status` ?

```
git status
```

გვაჩვენებს Repository-ის მიმდინარე მდგომარეობას.

2. დღევანდელი პრობლემა

წინა გაკვეთილზე შევქმენით:

```
my-first-git-project/  
index.html
```

შემდეგ გავუშვით:

```
git status
```

და Git-მა გვაჩვენა:

```
Untracked files:  
index.html
```

ანუ Git ხედავს `index.html` -ს, მაგრამ მისი ვერსია ჯერ შენახული არ არის.

დღეს უნდა ვისწავლოთ, როგორ გადავიდეთ აქედან:

```
შევცვალეთ ფაილი  
↓  
Git-მა დაინახა ცვლილება  
↓  
ავირჩიეთ შესანახი ცვლილება  
↓  
შევინახეთ ვერსია
```

ამისთვის ორი უმნიშვნელოვანესი ბრძანება გვჭირდება:

```
git add  
git commit
```

3. Git-ის სამუშაო პროცესი

Git-ის მუშაობა შეგვიძლია სამ ძირითად ნაწილად წარმოვიდგინოთ:

```
Working Directory  
↓
```

```
git add
↓
Staging Area
↓
git commit
↓
Repository
```

ეს პროცესი კარგად უნდა გავიგოთ, რადგან Git-ში მუშაობისას მუდმივად გამოვიყენებთ.

4. Working Directory

Working Directory არის ჩვენი პროექტის სამუშაო მდგომარეობა — ფაილები, რომლებზეც ახლა ვმუშაობთ.

მაგალითად:

```
school-website/
|
├─ index.html
├─ style.css
└─ script.js
```

თუ `index.html` გავხსენით და დავამატეთ:

```
<h1>My School</h1>
```

ეს ცვლილება ჯერ მხოლოდ Working Directory-შია.

Git ცვლილებას დაინახავს, მაგრამ ჯერ არ შეგვინახავს ის ახალ ვერსიად.

5. ფაილის მდგომარეობები Git-ში

დღეს სამი მნიშვნელოვანი მდგომარეობა გვჭირდება:

Untracked

ფაილი ახალია და Git ჯერ არ აკონტროლებს.

მაგალითად:

```
Untracked:
index.html
```

Modified

Git უკვე იცნობს ფაილს, მაგრამ მისი შენახული ვერსიის შემდეგ ფაილში ახალი ცვლილება გააკეთეთ.

მაგალითად:

```
Modified:
index.html
```

Staged

ფაილის ცვლილება უკვე ავირჩიეთ შემდეგ Commit-ში შესანახად.

```
Staged:
index.html
```

ამიტომ შეიძლება დავიმახსოვროთ:

```
Untracked / Modified
  ↓
git add
  ↓
Staged
  ↓
git commit
  ↓
Saved
```

6. რა არის Staging Area?

Staging Area არის ადგილი, სადაც ვამზადებთ ცვლილებებს შემდეგ Commit-ში შესანახად.

მაგალითად, ერთდროულად შევცვალოთ:

```
index.html
style.css
```

```
script.js
```

მაგრამ ახლა მხოლოდ `index.html`-ის ცვლილების შენახვა გვინდა.

ამის საშუალებას სწორედ Staging Area გვაძლევს.

ვამატებთ:

```
git add index.html
```

ახლა `index.html` მზადაა Commit-ისთვის, დანარჩენი ორი კი ჯერ არა.

მარტივი ანალოგია:

Working Directory არის სამუშაო მაგიდა.

Staging Area არის ყუთი, სადაც ვაწყობთ იმას, რისი შენახვაც გვინდა.

Commit კი ამ ყუთის შიგთავსის კონკრეტულ ვერსიად შენახვაა.

7. git add

Staging Area-ში ფაილის გადასატანად გამოიყენება:

```
git add
```

მაგალითად გვაქვს:

```
index.html
```

ვწერთ:

```
git add index.html
```

ახლა Git-ს ვეუბნებით:

`index.html` -ში არსებული ცვლილება მოამზადე შემდეგ ვერსიაში შესანახად.

8. პრაქტიკული მაგალითი — git add

შევქმნათ:

```
git-practice/  
  index.html
```

index.html :

```
<!DOCTYPE html>  
<html>  
<head>  
  <title>My Website</title>  
</head>  
<body>  
  <h1>My Website</h1>  
</body>  
</html>
```

თუ Repository ჯერ არ გვაქვს:

```
git init
```

შემდეგ:

```
git status
```

დავინახავთ დაახლოებით:

```
Untracked files:  
  index.html
```

ახლა:

```
git add index.html
```

და კვლავ:

```
git status
```

ამჯერად index.html გამოჩნდება **Changes to be committed** ნაწილში.

ანუ ფაილი უკვე **Staged** მდგომარეობაშია.

9. git add .

თუ პროექტში ბევრი ფაილი გვაქვს:

```
index.html  
about.html  
style.css  
script.js
```

შეგვიძლია თითოეული ცალ-ცალკე დავამატოთ:

```
git add index.html  
git add about.html  
git add style.css  
git add script.js
```

მაგრამ უფრო მარტივია:

```
git add .
```

წერტილი `.` ამ შემთხვევაში გულისხმობს მიმდინარე პროექტში არსებულ შესაბამის ცვლილებებს.

ანუ პრაქტიკულად:

```
git add .
```

→ მიმდინარე საქალაქეში ცვლილებებს Staging Area-ში ამზადებს.

განსხვავება

ერთი ფაილი:

```
git add index.html
```

ყველა მიმდინარე ცვლილება:

```
git add .
```

10. `git status` ხშირად უნდა გამოვიყენოთ

Git-ის სწავლისას განსაკუთრებით კარგი ჩვევაა ყოველი მნიშვნელოვანი მოქმედების შემდეგ:

```
git status
```

მაგალითად:

```
git status
```

```
git add index.html
```

```
git status
```

პირველ შემთხვევაში შეიძლება იყოს:

```
Untracked files:
  index.html
```

მეორეში:

```
Changes to be committed:
  new file: index.html
```

ამით მოსწავლე ვიზუალურად ხედავს, რა შეცვალა `git add`-მა.

11. რა არის Commit?

ახლა ფაილი Staging Area-ში გვაქვს, მაგრამ ცვლილება ჯერ კიდევ არ შეგვინახავს პროექტის ისტორიაში.

ამისთვის გამოვიყენება:

```
git commit
```

Commit არის პროექტის ცვლილებების შენახული ვერსია.

შეგვიძლია წარმოვიდგინოთ როგორც პროექტის კონკრეტული მომენტის **snapshot**.

მაგალითად:

```
Commit 1
"Create homepage"

    ↓

Commit 2
"Add navigation"

    ↓

Commit 3
"Add contact form"
```

Git ამ Commit-ებით პროექტის ისტორიას ქმნის.

12. Commit Message

Commit-ის შექმნისას უნდა აღვწეროთ, რა შევცვალეთ.

ამისთვის ვიყენებთ:

```
git commit -m "Create homepage"
```

აქ:

```
git commit
```

ქმნის Commit-ს.

`-m` ნიშნავს, რომ Commit-ს შეტყობინებას — **message-ს** ვაძლევთ.

ხოლო:

```
"Create homepage"
```

არის Commit Message.

13. კარგი და ცუდი Commit Message

Commit Message უნდა აღწერდეს ცვლილებას.

ცუდი:

```
git commit -m "changes"
```

```
git commit -m "stuff"
```

```
git commit -m "asdf"
```

ამ შეტყობინებებიდან მომავალში ვერ გავიგებთ, რა შევცვალეთ.

უკეთესი:

```
git commit -m "Create homepage"
```

```
git commit -m "Add navigation menu"
```

```
git commit -m "Add contact form"
```

```
git commit -m "Fix button style"
```

ანუ Commit Message უნდა იყოს მოკლე და გასაგები.

14. პირველი Commit — სრული მაგალითი

გვაქვს:

```
git-practice/  
index.html
```

პირველად:

```
git status
```

შემდეგ:

```
git add index.html
```

გამოწმებთ:

```
git status
```

და ვინახავთ:

```
git commit -m "Create homepage"
```

ახლა ჩვენი პირველი ვერსია შენახულია Git-ის ისტორიაში.

სრული პროცესი:

```
index.html
↓
git add index.html
↓
Staging Area
↓
git commit -m "Create homepage"
↓
Repository
```

15. მეორე ცვლილების გაკეთება

ახლა index.html -ში გვაქვს:

```
<h1>My Website</h1>
```

დავამატოთ:

```
<p>Welcome to my website!</p>
```

ფაილი გახდება:

```
<h1>My Website</h1>
<p>Welcome to my website!</p>
```

ახლა:

```
git status
```

Git უკვე გვაჩვენებს, რომ `index.html` შეიცვალა.

ეს არის:

Modified

ანუ:

Git უკვე იცნობს ამ ფაილს, მაგრამ მისი ბოლო Commit-ის შემდეგ ფაილში ცვლილება განხორციელდა.

16. მეორე Commit

ახლა ისევ იგივე პროცესი უნდა გავიაროთ.

```
git add index.html
```

შემდეგ:

```
git status
```

და:

```
git commit -m "Add welcome text"
```

ახლა ისტორიაში გვაქვს:

```
Commit 1
Create homepage
```

↓

```
Commit 2
Add welcome text
```

ეს უკვე არის ვერსიების კონტროლი პრაქტიკაში.

17. მესამე ცვლილება

დავამატოთ ღილაკი:

```
<button>Contact Me</button>
```

ახლა:

```
<h1>My Website</h1>

<p>Welcome to my website!</p>

<button>Contact Me</button>
```

ვამოწმებთ:

```
git status
```

შემდეგ:

```
git add .
```

და:

```
git commit -m "Add contact button"
```

ახლა გვაქვს სამი Commit:

```
Create homepage
  ↓
Add welcome text
  ↓
Add contact button
```

18. `git log` — Commit-ების ისტორია

როგორ ვნახოთ ჩვენ მიერ შექმნილი Commit-ები?

ამისთვის არსებობს:

```
git log
```

ის გვაჩვენებს Commit-ების ისტორიას.

მაგალითად:

```
commit a7f2...
Author: Student
Date: ...

    Add contact button

commit b8c1...
Author: Student
Date: ...

    Add welcome text

commit c9d4...
Author: Student
Date: ...

    Create homepage
```

აქ შეგვიძლია ვნახოთ:

- Commit-ის უნიკალური ID;
- ავტორი;
- თარიღი;
- Commit Message.

19. Commit ID

ყველა Commit-ს აქვს თავისი უნიკალური კოდი.

მაგალითად:

```
a72f51c863...
```

ამას ეწოდება:

Commit Hash / Commit ID

მაგალითად:

```
a72f51c → Add contact button  
c91ab21 → Add welcome text  
f812bc2 → Create homepage
```

Git ამ ID-ით არჩევს კონკრეტულ Commit-ს.

ამ ეტაპზე მოსწავლეებს მხოლოდ უნდა ესმოდეთ, რომ ყველა **Commit-ს** თავისი უნიკალური იდენტიფიკატორი აქვს.

20. git log --oneline

`git log` ზოგჯერ ბევრ ინფორმაციას გვაჩვენებს.

უფრო მოკლე ისტორიის სანახავად შეგვიძლია:

```
git log --oneline
```

მაგალითად:

```
a72f51c Add contact button  
c91ab21 Add welcome text  
f812bc2 Create homepage
```

ეს განსაკუთრებით მოსახერხებელია Commit-ების სწრაფად სანახავად.

21. მთლიანად Git Workflow

ახლა უკვე შეგვიძლია პირველი ორი გაკვეთილის სრული სამუშაო პროცესი გავაერთიანოთ.

პირველად ერთხელ ვქმნით Repository-ს:

```
git init
```

შემდეგ მუშაობისას:

```
ვცვლით კოდს  
↓  
git status  
↓  
git add .  
↓  
git status  
↓  
git commit -m "Describe changes"  
↓  
git log --oneline
```

ეს არის Git-ში მუშაობის ერთ-ერთი ძირითადი პროცესი.

22. ძალიან მნიშვნელოვანი — `git add` და `git commit` ერთი და იგივე არ არის

ეს მოსწავლეებს ხშირად ერევათ.

`git add`

```
git add index.html
```

არ ქმნის ახალ ვერსიას.

იგი მხოლოდ ამზადებს ცვლილებას შემდეგ Commit-ში შესანახად.

`git commit`

```
git commit -m "Add homepage"
```

ქმნის შენახულ ვერსიას Git-ის ისტორიაში.

ამიტომ:


```
git add
```

↓

ვირჩევთ ცვლილებებს

```
git commit
```

↓

ვინახავთ არჩეულ ცვლილებებს

23. პრაქტიკული სავარჯიშო 1 — შექმენი სამი Commit

მოსწავლემ შექმნას:

```
my-profile/  
index.html
```

ეტაპი 1

ჩაწეროს:

```
<h1>My Profile</h1>
```

შემდეგ:

```
git init  
git status  
git add .  
git commit -m "Create profile page"
```

ეტაპი 2

დაამატოს:

```
<p>My name is Nika.</p>
```

შემდეგ დამოუკიდებლად შექმნას Commit:

```
git status  
git add .  
git commit -m "Add profile information"
```

ეტაპი 3

დაამატოს:

```
<button>Contact Me</button>
```

და შექმნას მესამე Commit:

```
git add .  
git commit -m "Add contact button"
```

ბოლოს:

```
git log --oneline
```

უნდა ჰქონდეს დაახლოებით:

```
xxxxxxx Add contact button  
xxxxxxx Add profile information  
xxxxxxx Create profile page
```

24. სავარჯიშო 2 — დაალაგე სწორი თანმიმდევრობით

მოცემულია:

```
git commit -m "Add title"  
git status  
git add index.html
```

დაალაგეთ სწორი თანმიმდევრობით.

პასუხი:

```
git status  
git add index.html  
git commit -m "Add title"
```

25. სავარჯიშო 3 — რომელი ბრძანება გვჭირდება?

1. მინდა ვნახო პროექტის მიმდინარე მდგომარეობა.

?

2. მინდა `style.css` მოვამზადო შემდეგ Commit-ში შესანახად.

?

3. მინდა ყველა ცვლილება მოვამზადო Commit-ისთვის.

?

4. მინდა შევქმნა Commit შეტყობინებით `"Add navigation"`.

?

5. მინდა ვნახო Commit-ების ისტორია მოკლე ფორმით.

?

პასუხები

```
git status
```

```
git add style.css
```

```
git add .
```

```
git commit -m "Add navigation"
```

```
git log --oneline
```

26. სავარჯიშო 4 — განსაზღვრე ფაილის მდგომარეობა

სიტუაცია 1

ახლაზან შევქმენით:

```
about.html
```

მაგრამ `git add` არ გამოგვიყენებია.

მდგომარეობა: `Untracked`

სიტუაცია 2

`index.html` უკვე გვექონდა `Commit`-ში, მაგრამ მასში ახალი `<p>` დავამატეთ.

მდგომარეობა: `Modified`

სიტუაცია 3

შევასრულეთ:

```
git add index.html
```

მდგომარეობა: `Staged`

სიტუაცია 4

შევასრულეთ:

```
git commit -m "Update homepage"
```

ცვლილება უკვე **Repository**-ის ისტორიაშია შენახული **Commit**-ის სახით.

27. სავარჯიშო 5 — იპოვე შეცდომა

მოსწავლემ უნდა ახსნას, რა აკლია.

გვინდა ცვლილების შენახვა და პირდაპირ ვწერთ:

```
git commit -m "Add new paragraph"
```

მაგრამ ცვლილება `Staging Area`-ში არ გადაგვიტანია.

რა დაგვაფიქვდა?

პასუხი

ჯერ:

```
git add .
```

შემდეგ:

```
git commit -m "Add new paragraph"
```

ანუ:

```
git add .  
git commit -m "Add new paragraph"
```

28. საბოლოო პრაქტიკული დავალება

მოსწავლემ დამოუკიდებლად შექმნას პატარა ვებგვერდი და **მინიმუმ 3 Commit** გააკეთოს.

პროექტი:

```
student-website/  
|  
├─ index.html  
└─ style.css
```

Commit 1

შექმნას სათაური:

```
<h1>My Website</h1>
```

Commit:

```
Create homepage
```

Commit 2

დაამატოს ტექსტი:

```
<p>Welcome to my website.</p>
```

Commit:

```
Add welcome text
```

Commit 3

დაამატოს CSS, მაგალითად:

```
body {  
    font-family: Arial;  
}  
  
h1 {  
    text-align: center;  
}
```

Commit:

```
Add basic styles
```

ბოლოს შეასრულოს:

```
git log --oneline
```

და უნდა დაინახოს თავისი **სამი Commit**.